

sg-layout is a zero-dependency fractal window management framework

Version: v0.11.25 Architecture Brief

Technology: Vanilla JS + Web Components

Core Responsibility: Strict layout management (placing, sizing, tabbing, resizing, and moving panels).

Host Agnostic: Seamlessly wraps tools, apps, other sg-layout instances, or plain `<div>` elements.

The framework enforces a strict boundary around layout execution

What It Controls

- ✓ Placing panels
- ✓ Sizing components
- ✓ Moving windows
- ✓ Tabbing interfaces
- ✓ Resizing grids

What It Is NOT

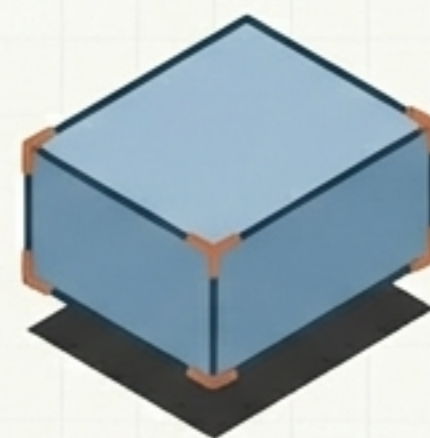
- Router (host app responsibility)
- State Manager
- Theme System (consumes but does not define tokens)
- Animation Library
- Drag-and-drop Library (only handles panel rearrangement)
- Virtual DOM (surgical DOM updates only)
- Command Bus
- Identity Provider

Four core principles define the system architecture



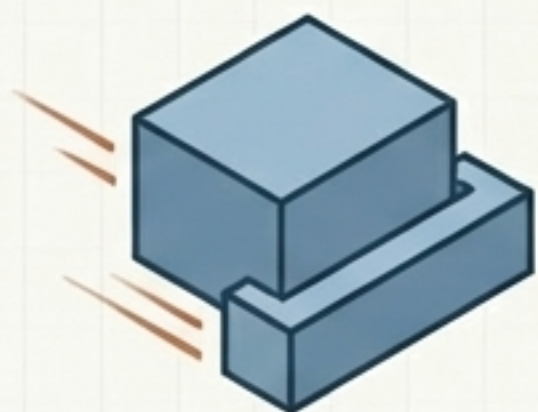
1. Zero Coupling

Hosted components do not import `sg-layout`. No name referencing.



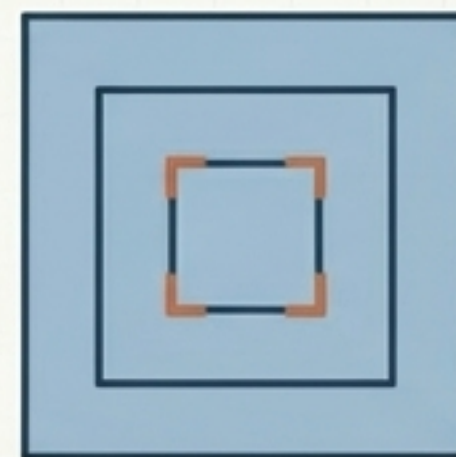
2. Shadow DOM Everywhere

True CSS isolation prevents leakage between framework chrome and hosted content. Structural enforcement blocks upward DOM querying.



3. Synchronous Bootstrap

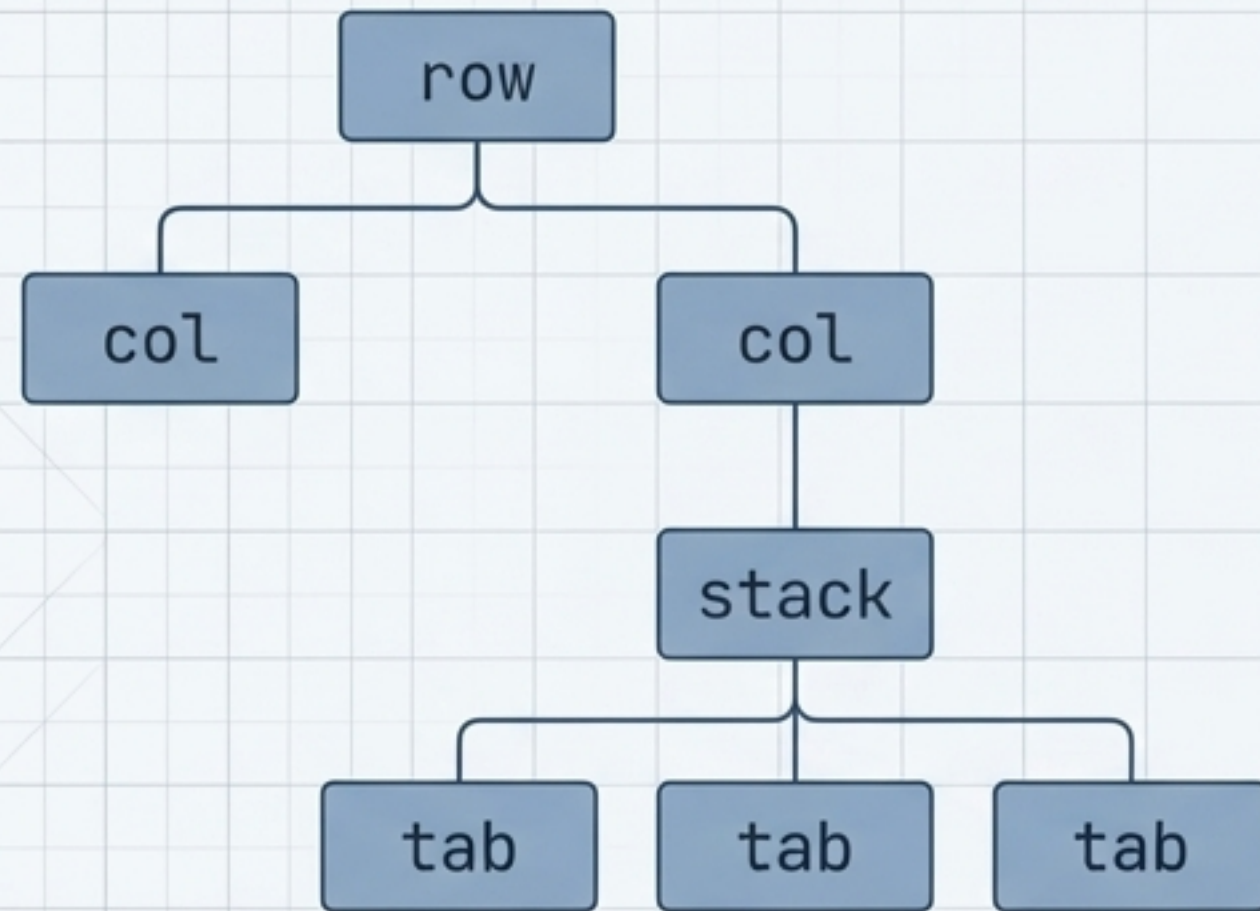
Layout construction and registration complete within a single call stack, before the first frame paint.



4. Fractal Composition

Layouts exist inside layouts. Outer layouts treat inner layouts exactly like any other hosted component.

The DOM is merely a projection of the underlying layout tree



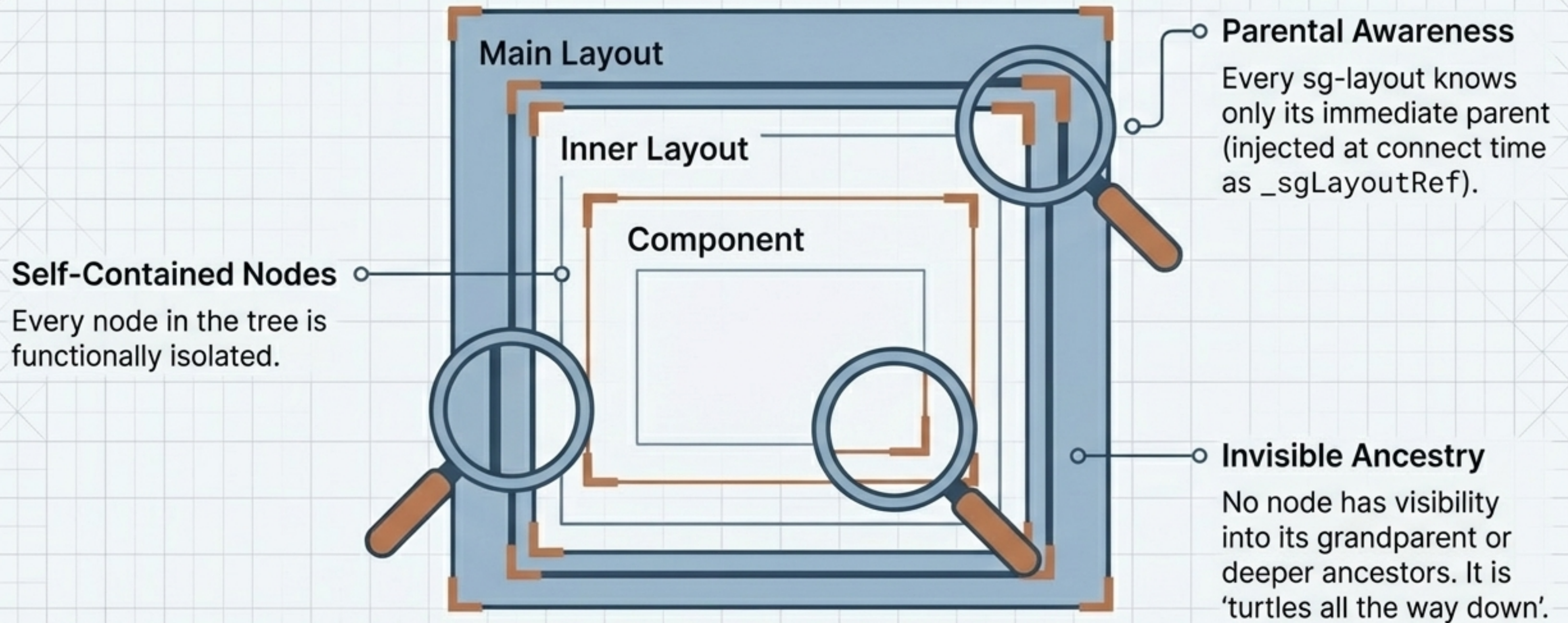
The Source of Truth

Internal state is a recursive tree of rows, columns, and stacks. Sizes are stored as normalised fractions that always sum to 1.0. They recalculate instantly on every resize drag.

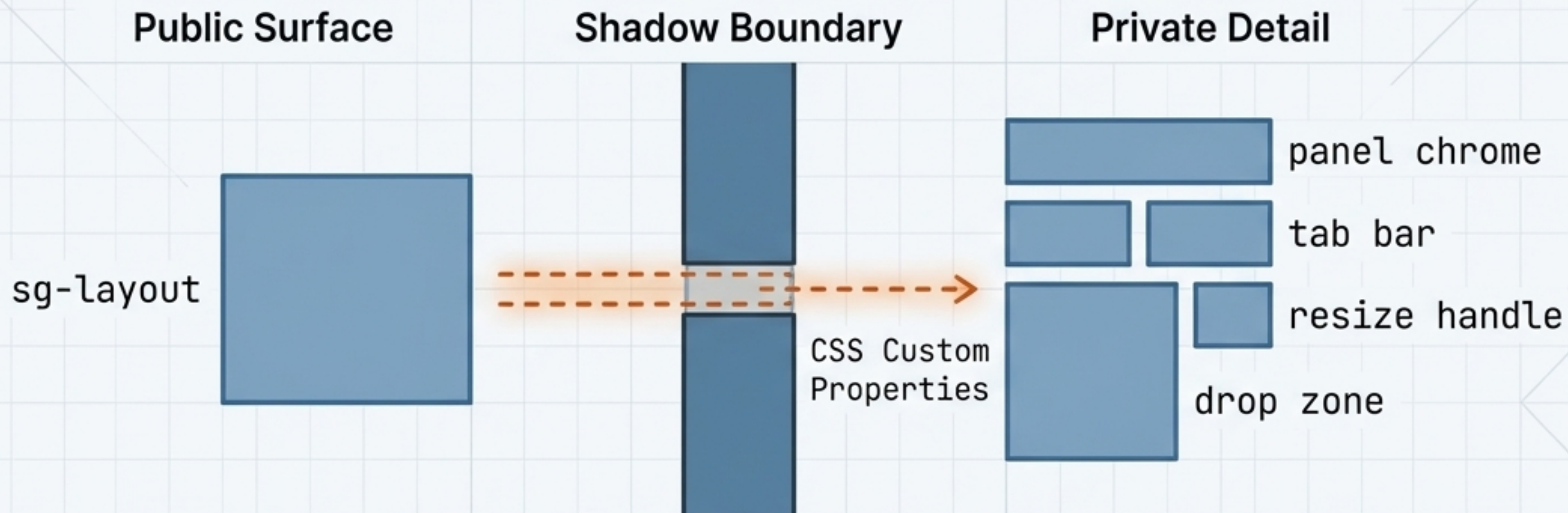
Architectural Contract

The DOM is never queried to understand layout state. When the tree changes, the affected subtree is surgically re-rendered.

Outer layouts treat inner layouts exactly like any other component

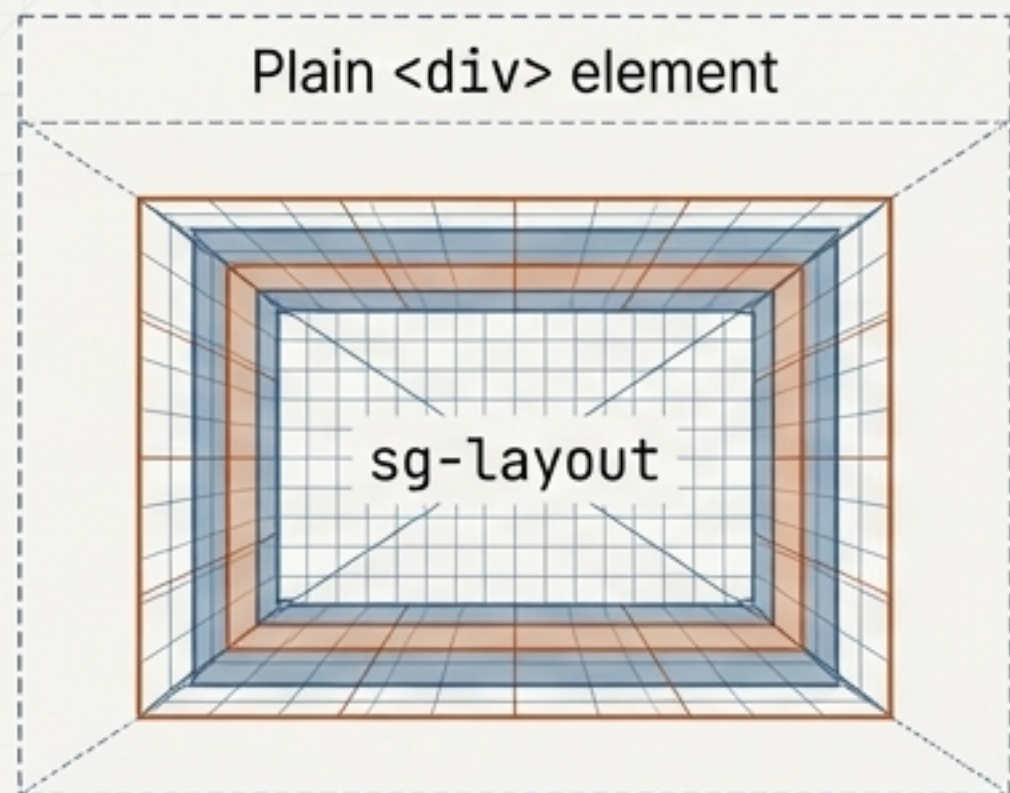


CSS custom properties are the exclusive bridge across the shadow boundary

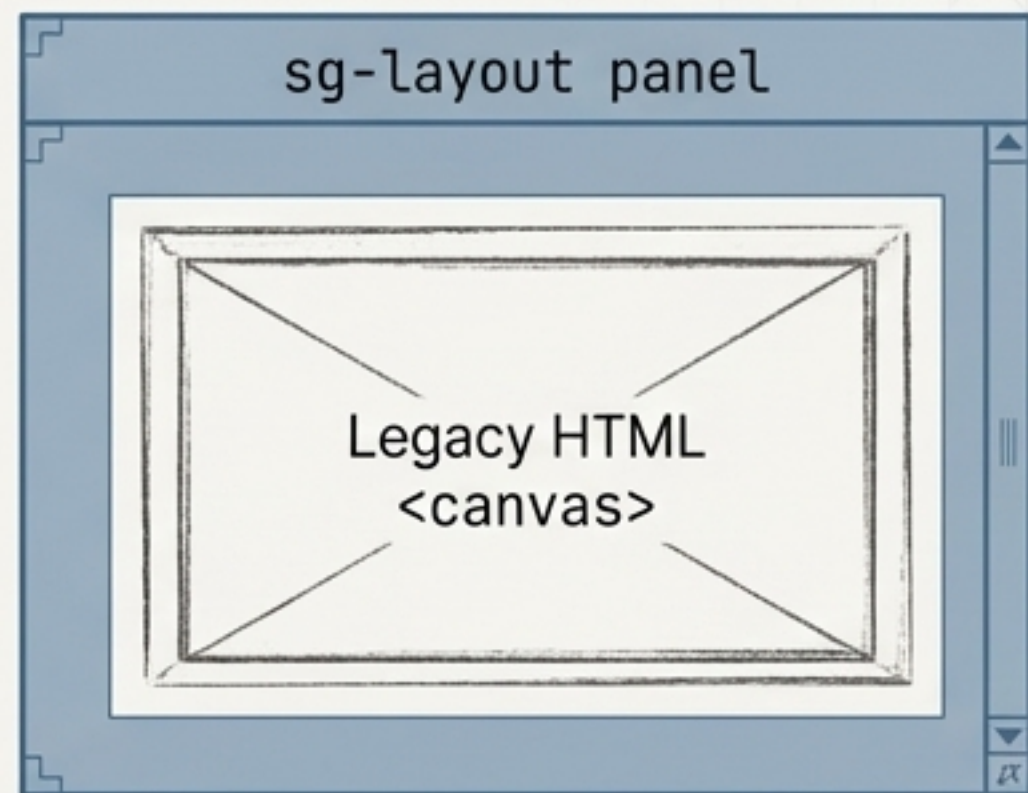


- **Public Surface:** `sg-layout` is the only custom element exposed publicly.
- **Private Detail:** All internal elements live purely inside `sg-layout`'s Shadow DOM.
- **Theme Contract:** No class-based overrides are supported. All visual customisation happens purely via CSS custom properties.

Bidirectional hosting seamlessly integrates plain HTML and framework instances



Direction 1: sg-layout inside plain HTML. It auto-bootstraps as root if no layout ancestor responds to its composed bubble event.

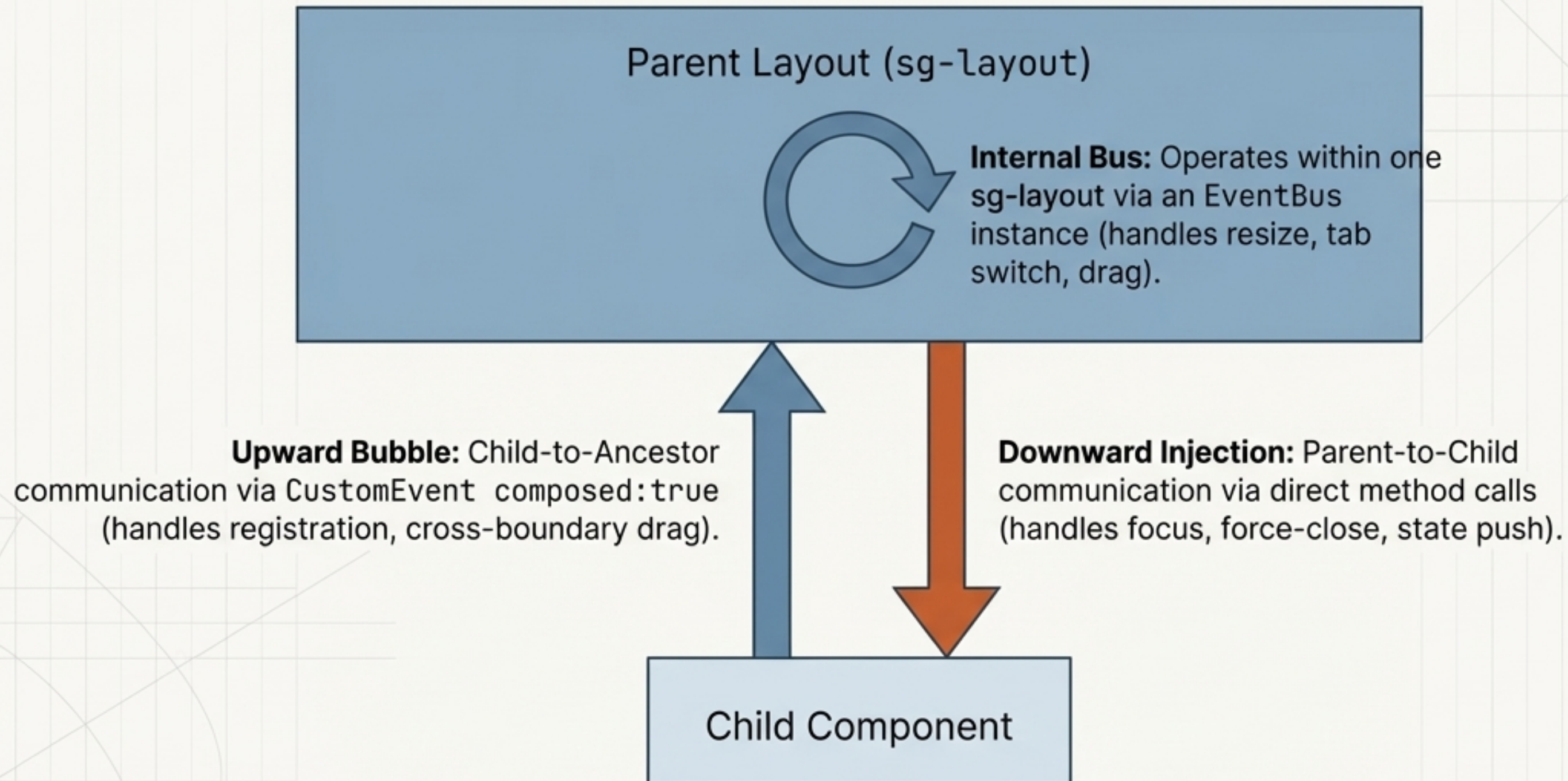


Direction 2: Plain HTML inside sg-layout. It simply does not participate in the registration protocol, but panel chrome and resize handles still work perfectly.

Architectural Contract Callout

The panel = **maximised** div rule: Any hosted element setting width: 100%; height: 100% must fill its panel correctly at all times, including after resize or tab switches.

Component communication flows through three distinct, controlled channels



The synchronous registration protocol guarantees deterministic bootstrapping

Single Call Stack Block



The Trigger: `sg-sg-layout:register` fires synchronously inside `connectedCallback`.

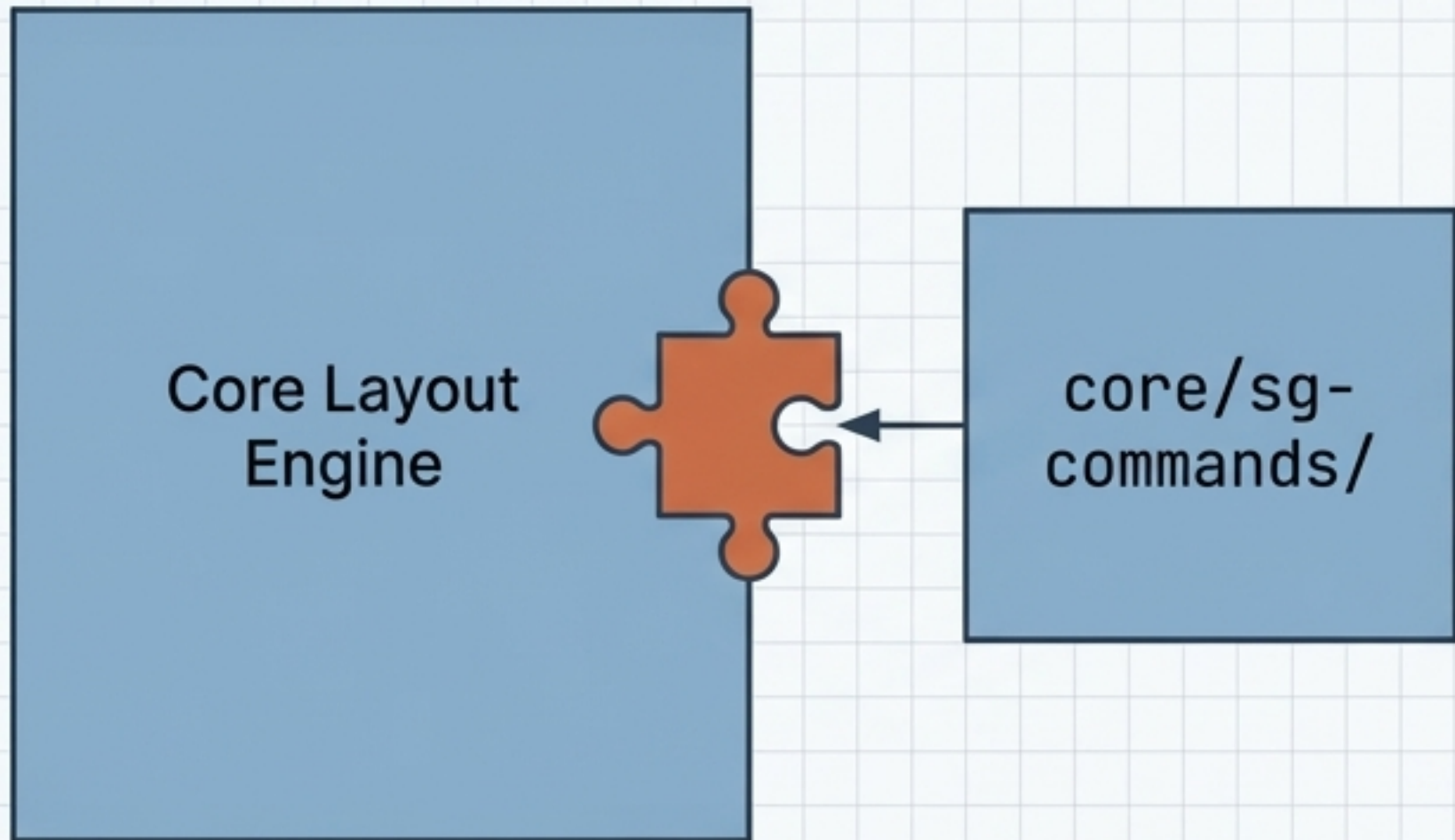


Fractal Wiring: Inner layout captures its own components (`stopPropagation` prevents upward leaking); outer layout captures the inner layout.



Infinite Loop Detection Signature: Call stack depth is monitored. If a second register event fires synchronously inside the first handler, and the element reference is already in the layout's tracked Set, it logs and breaks the cycle.

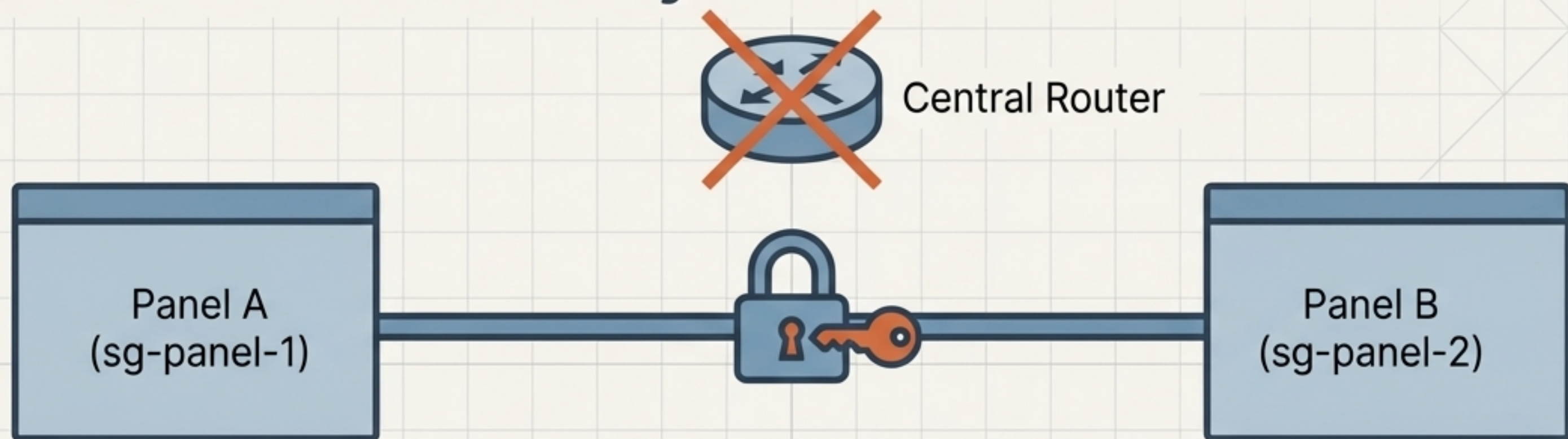
The companion command bus independently handles application-level messaging



Decoupled Architecture: sg-layout and sg-commands are fully independent. Commands route via exact panelId or broadcast to all panels.

Command	Function
show	Make panel visible
hide	Hide panel
focus	Bring tab to front
css-update {prop: value}	Set CSS custom properties
js-exec	Explicitly exposed methods only
state-push	Deliver data to the component
theme-change	Broadcast theme switch

Cryptographic keypairs enforce rogue component containment and identity



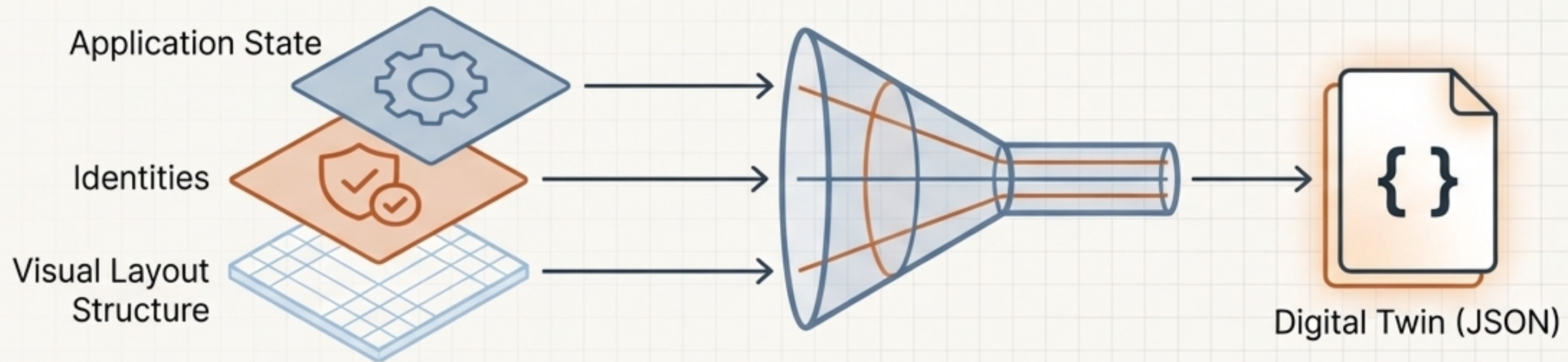
PKI Per Panel: `core/sg-panel-identity/` generates an ECDH P-256 keypair asynchronously for every panel instance during registration.

Unique Identity: Two panels hosting identical tags generate different keys.

Encrypted Point-to-Point: Messages encrypted to a panel's public key bypass routing tables—cryptography structurally enforces delivery (compatible with Web Workers).

Public Key Registry: Stored as `Map<panelId, CryptoKey>`, read-only from outside the framework.

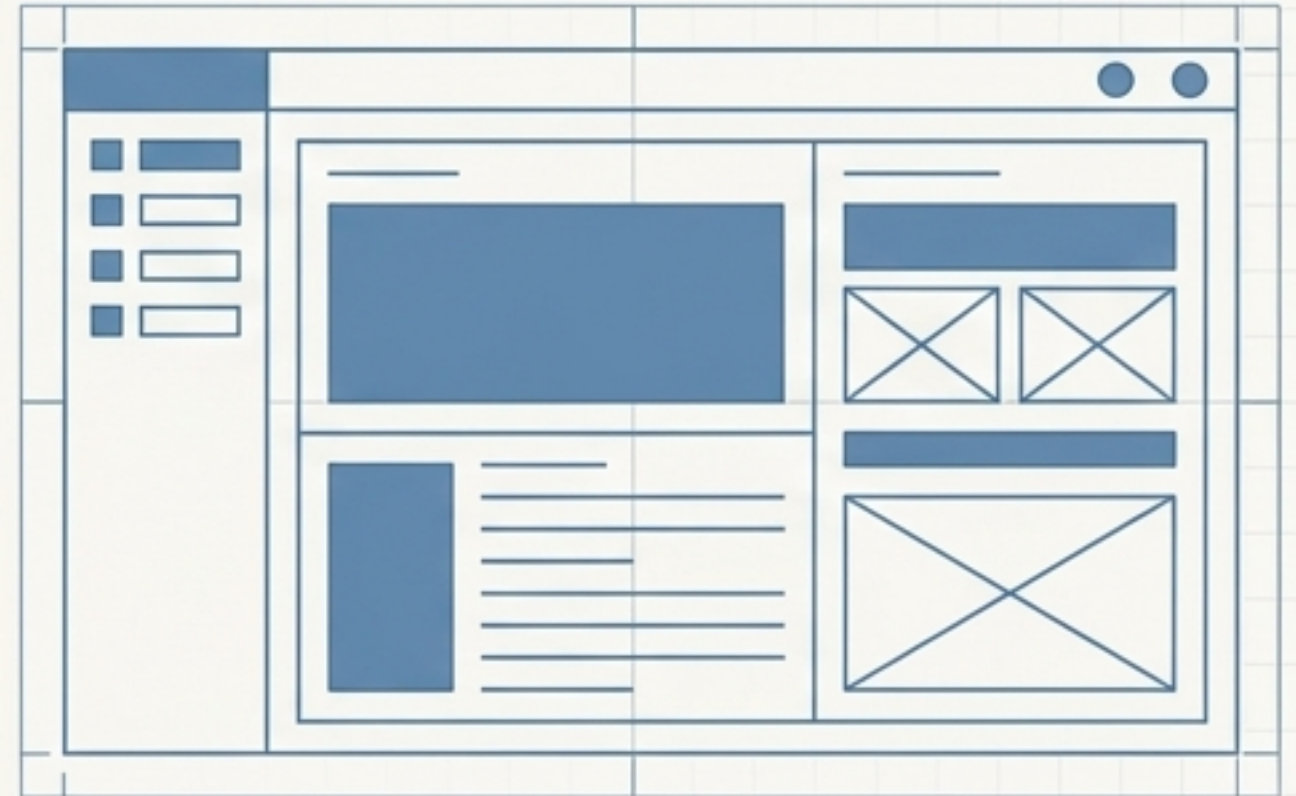
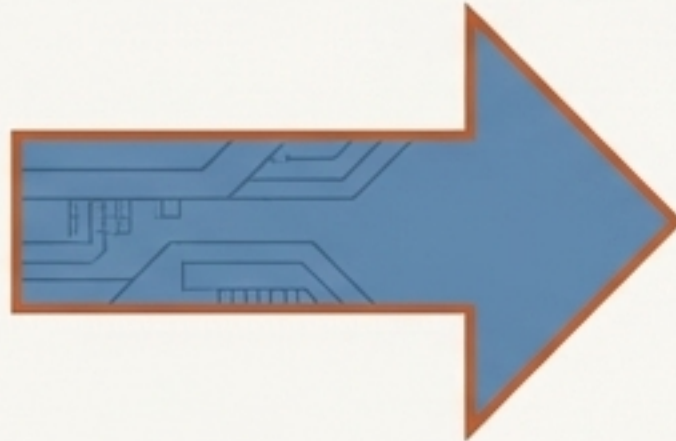
The entire structural tree and application state serialise into a digital twin



- The **Pull Protocol**: Framework requests state via `getLayoutStateAsync()` or `getLayoutState()`. Components natively falling back to `{}` if unimplemented.
- **Fractal Serialisation Rule**: Each layer is uniquely responsible for serialising exactly its own state. It asks its immediate children for theirs; grandchildren are invisible.
- **Output**: An infinitely recursive, fully deterministic JSON document capturing the exact moment in time.

A path-based mini DSL provides precise programmatic layout construction

```
SgLayout.build().row([  
  [...  
  ...]  
]).setLayout(root)
```



XPath-Inspired Addressing:

Navigate layouts programmatically (e.g., `row[0]/col[1]/stack[0]`).

DSL Builder API:

Utilise `SgLayout.build()` for rapid, highly readable construction.

Under the Hood:

The builder is purely a convenience wrapper generating the same strict JSON layout tree.

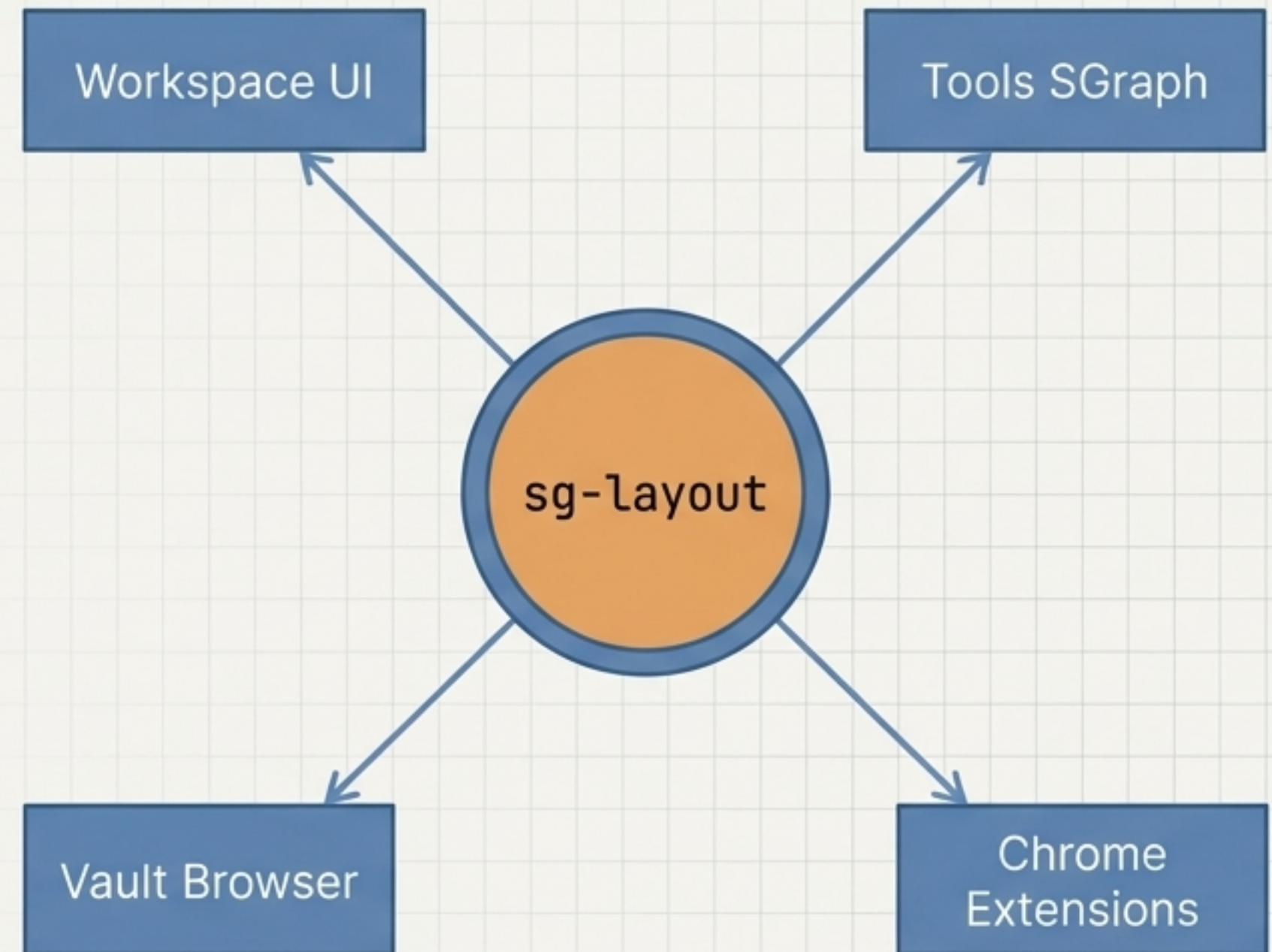
The framework operates as the core window manager across the ecosystem

Import Path:

```
import { SgLayout } from '/core/sg-layout/sg-layout.js'
```

Ecosystem Consumers:

- `sgraph_ai_app_send__ui__workspace` (replacing the rigid grid/collapse system).
- `tools.sgraph.ai` (flexible host for all tool interfaces).
- `sgraph_ai_app_send__ui__vault` (vault file browser layout).
- Chrome Extensions (popup panel layouts).



Key architectural parameters require resolution before the v0.1.0 kickoff

Performance

Stress test the 32x32 panel limit to determine exactly when CSS Grid must fall back to `position:absolute`.

Native OS Integration

Determine priority for popping panels out to native OS windows (via `window.open` + `postMessage` relay).

UX / Accessibility

Finalise touch UX for mobile resize handles and map ARIA roles/keyboard tab navigation across shadow boundaries.

Forward Compatibility

Design the Web Worker `postMessage` command bridge now to stabilise the API.